Отчёт об исследовании: недокументированный бинарный протокол поверх TCP

Английский оригинал: REPORT.md.

1. Контекст

Задача — восстановить формат сообщений недокументированного бинарного протокола поверх TCP, без спецификации, только по захватам, и сделать восстановление проверяемым. Результат сдачи — не утверждение, что формат понят, а набор явных описаний, каждое проверено против реальных байтов и каждое показывает, где оно падает.

Анализ работает по корпусу в `tests/corpus/`:

захват	сессий	байт потока	заметки
`corpus_capture_01.pcapng`	3	3036	трафик чтения и записи, обе команды
`corpus_capture_02.pcapng`	2	520	вторая, независимая запись
`corpus_capture_defects.pcapng`	1	74	несёт пропуск и неоднозначность
`synthetic_live.pcapng`	1	177	реальный обмен TCP с другой раскладкой

Каждый захват нормализуется к контракту захвата (`docs/CONTRACT.md`, `docs/schemas/capture.schema.json`) до анализа, поэтому модуль протокола никогда не трогает pcap напрямую. Для корпусного протокола заявлены два правила: `examples/corpus_rule_v1.json` и `examples/corpus_rule_v2.json`, оба валидны по `docs/schemas/rule.schema.json` и `docs/schemas/result.schema.json`.

Ограничения, определившие работу:

- Считаются только наблюдаемые байты. Пропуск в потоке записывается как диагностика и никогда не заполняется нулями, поэтому сообщение, пересекающее пропуск, сообщается как `incomplete`, а не угадывается.
- Совпадение на примерах — не доказательство. Каждое утверждение ниже проверено по всему корпусу, а контрпримеры сохраняются, а не отбрасываются.
- Изменение правила никогда не переписывает байты, против которых оно проверялось. Каждый вердикт несёт версию правила, которая его произвела.

Всё исследование воспроизводится через `python -m scripts.run_reference_investigation`, что пересобирает `docs/REFERENCE_INVESTIGATION.md` из захватов.

1.1 Предоставленные захваты

Собственный набор захватов организатора закоммичен в `tests/corpus/real/` и изучен тем же конвейером: три исследовательских захвата, один дополнительный и восемь захватов дефектов, плюс журнал действий. Это второй протокол, независимый от корпуса выше: двухбайтовая магия `0xa65c`, двухбайтовая длина тела, считающая четырёхбайтовый заголовок, счётчик сообщений и тип сообщения.

`python -m scripts.investigate_dataset` пишет `docs/DATASET_INVESTIGATION.md` из этих файлов. Оба направления фреймятся чисто на всех четырёх исследовательских захватах; первое чтение

поля длины (только нагрузка) производит шесть контрпримеров, которые чтение со смещением заголовка убирает. Каждый захват дефектов сообщает диагностику, для которой был построен, а раскодированные запросы коррелируются с журналом. Правила лежат в `examples/thermostand_request_v1.json`, `examples/thermostand_request_v2.json` и `examples/thermostand_response_v2.json`.

2. Методология

Работа следовала одному циклу, повторяемому до тех пор, пока правило не перестало меняться:

1. Прочитать поток и заявить раскладку как декларативное правило (`src/protocol/framing.py`, `src/protocol/rule.py`).
2. Применить правило к потоку (`src/protocol/engine.py`), произведя диапазоны и значения полей по каждому сообщению, каждое привязано к своим байтам провенансом.
3. Проверить правило по всему корпусу (`src/hypothesis/corpus.py`), классифицируя каждое сообщение `matched`, `mismatched`, `incomplete`, `ambiguous`, `uncovered`, `not_applicable`, `outdated` или `unknown`.
4. Взять `mismatched`-сообщения как контрпримеры, прочитать несомые ими значения и расширить правило (`src/hypothesis/versioning.py`).
5. Перепроверить и сравнить две версии.
6. Скоррелировать запросы с журналом действий (`src/hypothesis/journal.py`) и оценить альтернативные чтения любого поля, которое всё ещё предположение (`src/hypothesis/alternatives.py`).

Границы фрейминга решаются байтами, а не границами пакетов: сообщению разрешено пересекать несколько TCP-сегментов, и несколько сообщений могут делить один сегмент. Движок упорядочивает сегменты по номеру последовательности и трактует ретрансмиссию как лишний провенанс для тех же байтов.

3. Гипотезы и их основание

Каждая гипотеза ниже — вывод из наблюдения, а не догадка.

- **H1 (фрейминг).** Наблюдение: первые байты запроса — `01 00 00 01 10 01 00 ...`; байт 2 — `00`, байт 3 — `01`, и следующее сообщение начинается тремя байтами позже, совпадая с этой парой. Гипотеза: длина на смещении 2 — двухбайтовое big-endian поле, а сообщение — `header, payload`. Проверено в разделе 4; выжила.
- **H2 (команда).** Наблюдение: байт 0 принимает значение `01` в трафике чтения, но `02` и `03` в другом месте того же захвата. Гипотеза: байт 0 — код команды; первое правило допускало только `01`, что было слишком узко.
- **H3 (id параметра).** Наблюдение: байт 4 — фиксированное значение внутри сессии и отображается один-к-одному на параметр, который журнал называет для этого запроса. Гипотеза: байт 4 — идентификатор параметра (enum).
- **H4 (значение).** Наблюдение: байты 5 и 6 присутствуют только в запросах записи, и там они сдвигаются на небольшие величины между сообщениями. Гипотеза: байты 5–6 — big-endian значение, которое несут запросы записи.

H2 была полезной: именно слишком узкая её формулировка произвела контрпримеры, двинувшие всё уточнение.

Те же гипотезы выписаны по фиксированному шаблону в разделе 15 (`render_hypothesis_narrative`), заполненном из чисел корпуса, процитированных выше.

4. Решение о фрейминге

Поле длины — два байта на смещении 2, big-endian. Что оно считает, не записано, поэтому три кандидата были сфреймлены против каждого потока. Кандидат сохраняется только если он фреймит поток без остаточных байтов и без обрезанного последнего сообщения.

кандидат	потоков сфреймлено чисто	вердикт
`payload`	10 / 10	сохранён
`payload_and_length_field`	0 / 10	отвергнут, завышенные блоки
`entire_message`	0 / 10	отвергнут, останавливается после первого размера

По потокам, `corpus_capture_01` сессия 1 `A_to_B` — 300 байт: `payload` даёт 60 сообщений, 300/300 байт, чисто; два других дают 1 завышенный или 0 потреблённых байтов. Только `payload` фреймит все десять потоков. Это решение закреплено в обоих правилах как `"length_covers": "payload"`.

5. Контрпримеры

Правило v1 ограничивало `command` до `expected: [1]`. По `corpus_capture_01` оно произвело 80 контрпримеров, все одного вида: байт команды вне этого набора. Первые, с их координатами:

сессия	направление	смещение сообщения	поле	bytes_hex	провенанс
s2	A_to_B	0	command	`02000003130055`	packet_index 128, seq 20001, ts 1700000001.128991
s2	A_to_B	7	command	`020000031400ff`	packet_index 130, seq 20008, ts 1700000001.130990
s2	A_to_B	14	command	`0200000315009f`	packet_index 132, seq 20015, ts 1700000001.132990
s2	A_to_B	21	command	`0200000313005c`	packet_index 134, seq 20022, ts 1700000001.134990

Чтение значений в совокупности даёт повторяющуюся последовательность `13 14 15`, с `command = 02`. Сессия 3 несёт `command = 03`. Значит, наблюдаемый набор команд — `{1, 2, 3}`, и правило v1 ошибалась только в своём списке `expected`, а не в структуре. Каждый контрпример привязан к смещению внутри сообщения и к `packet_index` сегмента, из которого пришёл, поэтому читатель может найти его в pcap.

6. Уточнение и различение версий

Уточнение расширяет `command.expected` с `[1]` до `[1, 2, 3]`, давая правило v2 (`examples/corpus_rule_v2.json`). Применение обеих версий к одним байтам даёт:

метрика	v1	v2	дельта
matched	60	140	+80
mismatched	80	0	-80
контрпримеры	80	0	-80
точность	0.428571	1.0	+0.571429
плотность контрпримеров	57.142857	0.0	-57.142857
покрытие	0.5	0.5	+0.0

- `resolved_counterexamples`: 80 (каждый контрпример v1 совпадает у v2).
- `introduced_counterexamples`: 0 (v2 не ломает ничего, что v1 делала верно).
- `coverage_delta`: +0.0, потому что `coverage` считает применимые сообщения, а расширение ожидаемого набора не меняет, к каким сообщениям применима область; оно меняет только вердикт по ним.

Результат v1 остаётся на диске с `rule_version: 1`. Повторный прогон более нового правила его не перезаписывает; читатель всегда может сказать, какая версия произвела какой вердикт, а более старый результат помечается `outdated`, а не подаётся как текущий.

7. Перенос на второй захват

Правило v2 было применено без изменений к `corpus_capture_02.pcapng`, отдельной записи:

- сфреймлено сообщений: 80, счётчики: `matched=40`, `mismatched=0`, `not_applicable=40`;
- контрпримеры: 0.

Правило переносится на захват, под который не подгонялось. Это доказательство общности внутри корпусного протокола, а не доказанность её.

8. Граница применимости

Правило v2 затем было применено к `synthetic_live.pcapng`, захвату реального обмена TCP с намеренно другой раскладкой (little-endian длина, байт id транзакции, значения команд `0x21`/`0x22`/`0x23`):

- оба потока обрезаны при big-endian длине; 0/2 фреймятся чисто;
- счётчики: `incomplete=1`, `not_applicable=1`;
- контрпримеры: 1.

Правило не переносится. Его сбой записан, а не заглажен настройкой. Это граница применимости правила: оно описывает корпусный протокол, а раскладка `synthetic_live` требует другого правила.

9. Корреляция с журналом

Журнал действий для корпуса (`tests/corpus/corpus_journal.md`) записывает 185 транзакций со временем захвата каждого запроса. Сопоставление каждого сфреймленного запроса с записью журнала в ту же временную метку (окно 500 мс):

захват	запро-сов	скоррелиро-вано	несопоставленных за-просов	target = параметр журнала
corpus_capture_01	140	140	0	140 / 140
corpus_capture_02	40	40	0	40 / 40

Каждый запрос попадает на запись, и раскодированное поле `target` равно параметру, который называет журнал, на каждом скоррелированном запросе. Это независимое доказательство извне байтов, что байт 4 — идентификатор параметра (H3). Поле `value` сравнивается только там, где журнал записывает записанный результат; запросы чтения не несут значения, поэтому там знаменатель нулевой и совпадение не заявляется.

10. Альтернативные объяснения

Поскольку H2, H3 и H4 — предположения, конкурирующие чтения этих полей были оценены по сфреймленному корпусу. Балл — `support / (support + contradict)`.

Поле `command` (140 значений) — объявленный смысл — код команды:

чтение	поддержка	противоречий	балл
low_cardinality	140	0	1.00
offset_shift_+1	140	0	1.00
counter	137	2	0.99
constant	60	80	0.43
message_length	0	140	0.00

Поле `target` (140 значений): `low_cardinality` даёт 1.00; `constant` 0.14.

Поле `value` (60 значений): `constant` и `message_length` дают оба 0.02, то есть не объясняют почти ничего.

Результаты информативны именно потому, что не все благоприятны:

- `offset_shift_+1` даёт столько же, сколько `low_cardinality` для `command`. Байт на смещении 1 (`flags`) константен `0` по корпусу, поэтому константный сосед неотличим от поля низкой кардинальности. Кодировщик не может разделить их на этом корпусе и говорит об этом.
- `constant` даёт 0.43 для `command`, что и есть верное чтение того факта, что 60 из 140 запросов — чтения (`command = 1`).
- `counter` даёт 0.99, потому что id параметра циклится, а не потому что поле — счётчик. Высокий балл — кандидат, а не вывод.
- Ни одно чтение контрольной суммы не выжило ни на одном поле, поэтому ни одно не сообщается.

Поле `target` сохраняет `low_cardinality` как лучшее чтение, согласующееся с H3; поле `value` не объясняется ни одним проверенным чтением, поэтому остаётся помеченным предположением.

11. Применимость и открытые вопросы

Подтверждено внутри проверенной области:

- сообщение — `command, flags, length(big-endian, payload), payload`;
- наблюдаемые команды: `1` чтение, `2` запись, `3` измерение;
- байт 4 — id параметра; байты 5–6 несут значение при записи;

- правило переносится на `corpus_capture_02`.

Открыто:

- смысл байта `flags` не установлен; он константен `0` в корпусе, поэтому ничто не отличает «всегда ноль» от «всегда ноль здесь»;
- поле `value` не имеет выжившего объяснения и остаётся гипотезой;
- ответы (`B_to_A`) фреймятся, но не интерпретируются по полям;
- пропуск и неоднозначность захвата дефектов показываются как диагностики, но как сам протокол трактует потерянный сегмент — неизвестно.

12. Ограничения

- Восстановление — интерпретация, а не спецификация. Оно верно для проверенных захватов и молчит вне них.
- Вопрос фрейминга решён только для этого протокола; поток, чьё поле длины считает что-то другое, требует своего правила.
- Смыслы полей, помеченные `hypothesis: true`, не доказываются совпадением.
- Альтернативы берутся из фиксированного набора чтений; смысл вне этого набора этим анализом не был бы найден.
- Корпус был сгенерирован, а его генератор не выведен из реального устройства, поэтому корпусный протокол может не совпадать ни с одним развёрнутым.
- Окно открывает файл захвата, а меню проекта создаёт, открывает, сохраняет, экспортирует и импортирует каталог переносного проекта; командный `src.project.cli` ведёт тот же формат.

13. Проверка

Исследование и окно были прогнаны по эталонной выгрузке до демонстрации. Окно открывает чистый захват и перечисляет его три сессии; клик по байту сообщает его индекс пакета, номер последовательности и временную метку; применение правила v1 даёт 60 matched на `s1` и 60 mismatched на `s2`, а по всему захвату `matched=60, mismatched=80, not_applicable=140`; diff версий сообщает 80 разрешённых и 0 внесённых контрпримеров. Дефекты были найдены так и исправлены. Запись — в `docs/GUI_VERIFICATION.md`, а автоматическая форма того же сценария — `tests/ui/test_e2e.py` (четырнадцать тестов по реальным байтам и реальному движку).

13.1 Проверка на реальном корпусе

Набор организатора был получен после окна и разобран тем же конвейером. Это второй, независимый протокол (магия `0xa65c`, двухбайтовая длина, считающая четырёхбайтовый заголовок) по четырём исследовательским захватам и восьми диагностическим захватам с одним условием.

Проверка	Результат
Сессии против `dataset_manifest.json`	2, 1, 1, 1 — воспроизведены точно
Сообщений-запросов	45, все `matched`, 100 % покрытия байт
Сообщений-ответов	45, все `matched`, 100 % покрытия байт
Правило v1 (длина только нагрузки)	6 контрпримеров (5 mismatched, 10 incomplete)
Правило v2 (`length_bias: 4`)	0 контрпримеров
Корреляция с журналом	14 из 14 запросов скоррелированы на захвате конфигурации
Диагностики	каждый захват сообщает своё условие: `gap`, `ambiguity`, `checksum_offload`

Каждое поле сообщения термостенда было определено и сверено с журналом: магия, счётчик, тип, статус, блок конфигурации и массив измерений. Что не перенеслось без изменений — соглашение фрейминга: наивное чтение фреймило на четыре байта короче, и именно это контрпример, который цикл и призван вскрыть. Полная запись — в `docs/REAL_CORPUS_ANALYSIS.md`; сгенерированные числа по захватам — в `docs/DATASET_INVESTIGATION.md`.

14. Артефакты

артефакт	роль
`examples/corpus_rule_v1.json`	первое правило, со слишком узким набором команд
`examples/corpus_rule_v2.json`	уточнённое правило
`tests/corpus/corpus_capture_01.pcapng`	основной захват
`tests/corpus/corpus_capture_02.pcapng`	захват переноса
`tests/corpus/corpus_capture_defects.pcapng`	диагностики пропуска и неоднозначности
`tests/corpus/synthetic_live.pcapng`	граница применимости
`tests/corpus/corpus_journal.md`	журнал действий для корреляции
`docs/schemas/capture.schema.json`	контракт нормализованного захвата
`docs/schemas/rule.schema.json`	контракт правила
`docs/schemas/result.schema.json`	контракт результата
`scripts/run_reference_investigation.py`	пересобирает этот анализ
`docs/REFERENCE_INVESTIGATION.md`	сырой вывод того скрипта по разделам
`docs/demo.md`	проход по окну, одиннадцать шагов
`docs/demo_cli.md`	то же исследование из командной строки
`docs/DEFENSE_FAQ.md`	двадцать восемь вероятных вопросов защиты с их доказательствами
`docs/CORPUS_ANALYSIS.md`	числа корпуса за правилами, с командами их воспроизведения
`docs/RULE_V1_V2.md`	запись версий для двух корпусных правил
`docs/GUI_VERIFICATION.md`	что окно было проведено показать, и найденные дефекты
`docs/PRESENTATION_VERIFICATION.md`	каждое утверждение деки отображено на его доказательство
`presentation/slides.pdf`	дека, пятнадцать слайдов, с реальными снимками
`presentation/slide-light.pdf`	та же дека в светлой теме, пятнадцать слайдов
`presentation/slide.pptx`	та же дека как переносимый PPTX, одна отрисованная страница на слайд
`presentation/defense_script.md`	маршруты спикера на 5 и 10 минут к защите
`presentation/screenshots/`	одиннадцать захватов окна
`tests/ui/test_e2e.py`	сценарий как четырнадцать offscreen-тестов

Раздел выше производится `render_hypothesis_narrative`; числа — те же, что процитированы в разделе 3, а не второе измерение.

15. Гипотезы в повествовательной форме

Гипотеза 1: двухбайтовая big-endian длина на смещении 2 фреймит поток

Поле `length` на смещении 2. Первые байты запроса — 01 00 00 01 10 01 ...: байт 2 — 00, байт 3 — 01, и следующее сообщение начинается тремя байтами позже, совпадая с этой парой. С length_covers payload это фреймит все десять потоков корпуса без остаточных байтов; два других кандидата не фреймят ни один. Применимость: все десять потоков корпуса (corpus_capture_01 и corpus_capture_02). Не проверено на synthetic_live.

Гипотеза 2: байт 0 — код команды

Поле `command` на смещении 0. Байт 0 — 0x01 в трафике чтения, но 0x02 и 0x03 в другом месте того же захвата, поэтому правило v1, допускавшее только 0x01, было слишком узко. Проверка по корпусу на 140 сообщениях, matched 60, противоречий 80. Лучшее альтернативное чтение: entropy_enum (поддержка 140, противоречий 0, балл 1.0); в корпусе не опровергнуто. Применимость: команды 0x01 чтение, 0x02 запись, 0x03 измерение, по сессиям в corpus_capture_01. Не проверено на synthetic_live.

Гипотеза 3: байт 4 — идентификатор параметра (enum)

Поле `target` на смещении 4. Байт 4 фиксирован внутри сессии и отображается один-к-одному на параметр, который журнал называет для этого запроса. Проверка по корпусу на 140 сообщениях, matched 140, противоречий 0. Лучшее альтернативное чтение: low_cardinality (поддержка 140, противоречий 0, балл 1.0); в корпусе не опровергнуто. Применимость: 140 из 140 запросов совпадают с параметром журнала. Не проверено на synthetic_live.

Гипотеза 4: байты 5 и 6 несут записанное значение

Поле `value` на смещении 5. Байты 5 и 6 появляются только в запросах записи и сдвигаются на небольшие величины между сообщениями. Поле не несёт ожидаемого значения, поэтому оно присутствует, а не matched; ни одно конкурирующее чтение не выжило. Проверка по корпусу на 60 сообщениях. Лучшее альтернативное чтение: entropy_parameter (поддержка 60, противоречий 0, балл 1.0); в корпусе не опровергнуто. Применимость: записи в corpus_capture_01. Не проверено на synthetic_live.